

Lab 1 - Introduction to SIMD programming

Oguz Kaya

oguz.kaya@universite-paris-saclay.fr

Environment. In this lab, we will use Compiler Explorer (<https://godbolt.org>) to compile and run our programs, so that everybody works with the same compiler on the same processor type. Select the language C++ and a recent version of the x86-64 gcc compiler, and enter the compilation options given below in the “Compiler options” field. To run your program, copy-paste your code into the code field, then tick the “Execute the code” box in the “Output...” menu of the compiler pane; the output of the program appears in the “Executor” panel. Save your code regularly in a local file on your machine, since Compiler Explorer does not store it.

SIMD programming with AVX. In this lab, we will write our first SIMD programs using the AVX/AVX2 instruction sets, through the *intrinsic* functions provided by the compiler. An AVX register is 256 bits wide, and therefore holds 8 single-precision floating point numbers (float, 32 bits each). We will only use a handful of intrinsics in this lab: loading a vector from memory, storing a vector to memory, and the four basic arithmetic operations.

<code>__m256 x;</code>	declares a vector <code>x</code> of 8 floats (an AVX register)
<code>__m256 _mm256_loadu_ps(const float* p)</code>	load 8 floats from memory starting at <code>p</code>
<code>void _mm256_storeu_ps(float* p, __m256 v)</code>	store the 8 floats of <code>v</code> to memory starting at <code>p</code>
<code>__m256 _mm256_add_ps(__m256 a, __m256 b)</code>	element-wise addition
<code>__m256 _mm256_sub_ps(__m256 a, __m256 b)</code>	element-wise subtraction
<code>__m256 _mm256_mul_ps(__m256 a, __m256 b)</code>	element-wise multiplication
<code>__m256 _mm256_div_ps(__m256 a, __m256 b)</code>	element-wise division

These six functions are the only intrinsics you are allowed to use in this lab. Anything else you need (e.g., a vector filled with zeros or with a constant) must be built from them: for instance, by preparing a small array of 8 floats in memory and loading it.

These functions are declared in the header `<immintrin.h>`.

To compile with the AVX/AVX2 instruction sets, use the following compiler options (in the “Compiler options” field of Compiler Explorer, or on the command line if you compile with `g++` on your own machine):

`-O3 -fno-tree-vectorize -std=c++11 -mavx2`

The flag `-fno-tree-vectorize` prevents the compiler from vectorizing your scalar code automatically, so that the only SIMD instructions in your program are the ones you write yourself.

Part 1

Copying an array using SIMD

The goal of this exercise is to learn the basics of SIMD programming using AVX, by copying an array into another array using vectorized load/store instructions. A skeleton code is provided in the file `copy.cpp`; it already contains the timing and verification helpers, and marks with `TODO` the parts you have to fill in for each question.

Ex. 1

- Allocate two float arrays `A` and `B` of size N (divisible by 8), then initialize `A` such that `A[i] = i`.
- Implement a scalar (non-vectorized) function that copies the content of `A` to `B`.
- Implement a vectorized version that performs the same operation, copying 8 floats at each iteration with one `_mm256_loadu_ps` and one `_mm256_storeu_ps`. Verify that `B` is identical to `A` after the copy.
- What happens if N is not a multiple of 8? Modify your vectorized function so that it handles the remaining $N \bmod 8$ elements correctly using a scalar loop.
- Compare the runtime of both versions for 1000 consecutive executions, for $N = 1024$ and for $N = 2^{20}$. Does the vectorized version go faster than the scalar one? How much is the speedup for $N = 1024$ versus $N = 2^{20}$?

Vector inner product using SIMD

The goal of this exercise is to compute the inner product of two vectors x and y :

$$x^T y = \sum_{i=1}^N x_i y_i$$

using vectorization. A skeleton code is provided in the file `dot.cpp`, with the same structure as in the previous exercise.

Ex. 2

- a) Allocate two float arrays `x` and `y` of size N , then initialize them (`x[i] = i` and `y[i] = 1` for instance, so that the expected result is easy to verify).
- b) Write a scalar (non-vectorized) function that computes the inner product of `x` and `y`.
- c) Write a second version that performs the same operation with vectorized load/mul/add instructions. Use a vector accumulator `__m256 vsum`, the vector counterpart of the scalar variable `sum`, initialized to zero (remember that you only have the six intrinsics above at your disposal), and at each iteration multiply 8 elements of `x` with 8 elements of `y` and add the result to `vsum`. At the end of the loop, `vsum` contains 8 partial sums; store them into an array of 8 floats with `_mm256_storeu_ps`, then add these 8 values with a scalar loop to obtain the final result. Your function must also handle the case where N is not a multiple of 8, as in the previous exercise. Verify that the result matches the scalar version.
- d) Compare the runtime of both versions for 1000 consecutive executions, for $N = 1024$ and for $N = 2^{20}$. Does the vectorized version go faster than the scalar one? How much is the speedup for $N = 1024$ versus $N = 2^{20}$?
- e) Is the result of the vectorized version precisely the same as that of the scalar version in floating point arithmetic? Whether it is or not, can you explain why?